

Open-Vocabulary Object Detection and Visual Grounding

Full-split reproduction, evaluation, and analysis

Zheng Renjie Lin Hongyu He Jiayang Deng Haotian

Group 8 Southern University of Science and Technology

CS308 Final Project Defense June 2026



Reproducible benchmark comparison on COCO and RefCOCO

Roadmap

Main story

- ① Motivation and task scope
- ② Related work and model choice
- ③ Approach and evaluation pipeline
- ④ Full COCO and RefCOCO results
- ⑤ Ablations, qualitative analysis, and takeaways

What to remember

- Full-split results are the only headline numbers.
- RefCOCO uses all expressions, not just a diagnostic slice.
- The comparison covers accuracy, speed, memory, and robustness.

What Problem Are We Solving?

Task

Open-vocabulary detection asks a detector to localize arbitrary text prompts. Visual grounding goes one step further and asks for the exact instance described by a referring expression.

- Fixed-label detectors cannot directly handle new words without retraining.
- Grounding is harder than category detection because the model must resolve attributes, relations, and instance ambiguity.
- Our goal is not to train a new model, but to reproduce and compare strong pretrained baselines under one rigorous protocol.

Project Goal

Reproduce
Evaluate
Analyze
Explain

The final report and code answer both sides of the assignment: open-vocabulary detection and visual grounding.

Related Work and Model Choice

OWL-ViT

A compact text-image similarity detector. It is the lightest baseline in our comparison, but not the strongest on accuracy.

Grounding DINO

A grounding-oriented detector. It gives the best accuracy on both COCO and RefCOCO, so it serves as the accuracy-first reference.

YOLO-World

A real-time open-vocabulary detector. It is the best deployment-oriented baseline because it balances speed and recall well.

Why these three?

They span the accuracy-speed-memory spectrum, which matches the application-oriented scope of the project.

What We Built

Task coverage	Both open-vocabulary object detection and visual grounding are evaluated.
Model coverage	OWL-ViT, Grounding DINO, and YOLO-World are all reproduced with pretrained weights.
Data coverage	Full COCO val2017 and full RefCOCO val with all expressions are used for the final claims.
Metric coverage	AP / AP50 / AP75 / AR100, grounding Acc@0.5 / Acc@0.75 / mean IoU, FPS, and peak VRAM.
Extra analysis	Score-threshold sensitivity, OWL-ViT NMS, and qualitative examples are included.
Reproducibility	Shared config block, manifest caching, local RefCOCO parquet fallback, and timestamped server logs.

Important

The 100-row and 500-image results are diagnostics only. All headline claims in the report come from full-split runs.

Implementation Map

Model wrappers

- `evaluate_coco_*.py`: full COCO val2017 inference, predictions, timing, and VRAM.
- `evaluate_refcoco.py`: full RefCOCO val, all expressions, local parquet fallback.
- `evaluate_threshold_sensitivity.py`: re-scores saved COCO predictions under multiple thresholds.

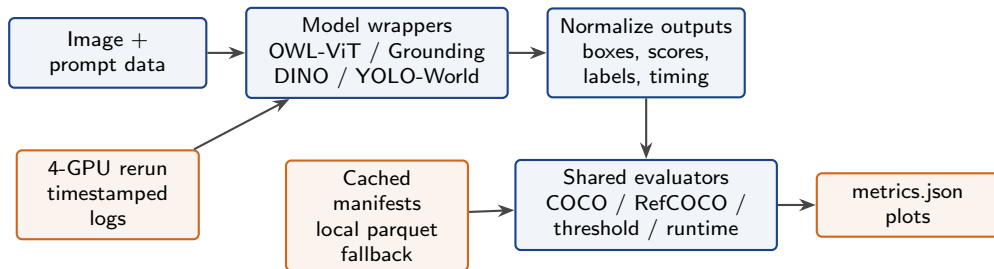
Shared infrastructure

- One output schema: image id, prompt id, score, box, runtime, and peak memory.
- `coco_eval_utils.py` and `generate_report_figures.py` keep metrics and plots consistent.
- `run_full_suite_4gpu_server.sh` orchestrates the 4-GPU rerun with timestamped logs.

Normalization idea

Converting all model APIs to one schema is what makes the comparison fair: same data, same metrics, same reporting path.

Unified Pipeline



- Same code path for accuracy, speed, and qualitative visualization.
- Same output directory layout for every full-split rerun.
- Cached manifests prevent silent changes in the benchmark data.

Experimental Setup

Hardware and software

- 4 × TITAN RTX server
- NVIDIA driver 550.120, CUDA 12.4
- PyTorch 2.6.0+cu124, Torchvision 0.21.0+cu124
- Transformers 5.5.3, Ultralytics 8.4.70

Formal evaluation

- COCO val2017: 5,000 images, 80 category prompts
- RefCOCO val: 8,811 rows, 25,080 expressions
- Diagnostics only: 100-row / 500-image subsets

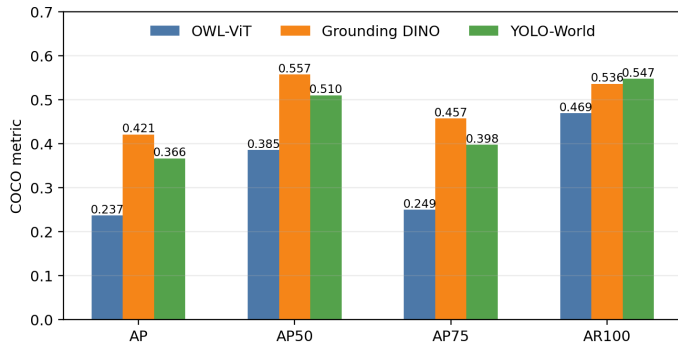
Metrics

- COCO: AP, AP50, AP75, scale AP, AR100
- RefCOCO: Acc@0.5, Acc@0.75, mean IoU
- Efficiency: pipeline FPS and peak allocated GPU memory
- Ablations: threshold sensitivity and NMS

What this means

The slides and report use only full-split numbers for the final claims. Smaller runs are retained for debugging and sanity checks.

COCO Detection Results



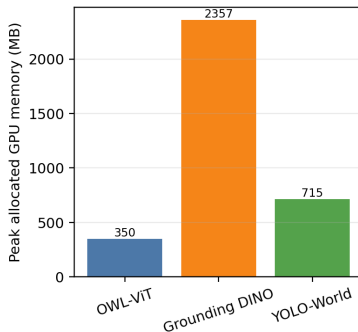
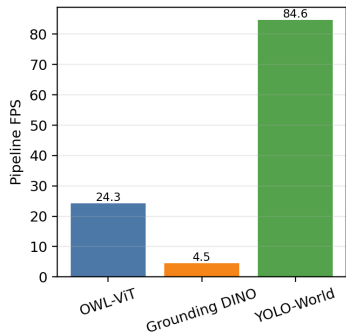
Key numbers

- Grounding DINO: AP 0.421, AP50 0.557, AP75 0.457, AR100 0.536.
- YOLO-World: AP 0.366, AP50 0.510, AP75 0.398, AR100 0.547.
- OWL-ViT: AP 0.237, AP50 0.385, AP75 0.249, AR100 0.469.

Interpretation

Grounding DINO leads on AP, AP50, and AP75. YOLO-World leads on AR100 and speed.

Efficiency and Deployment Trade-off



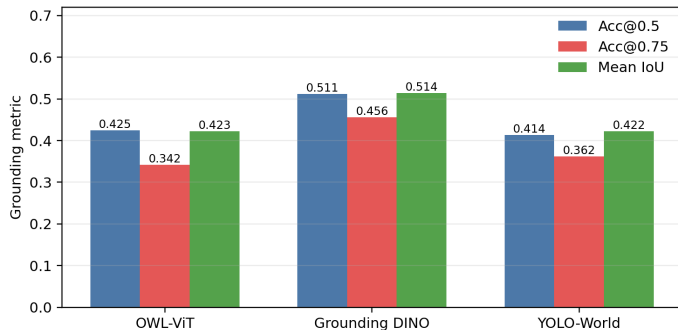
Pipeline timing

Model	FPS	MB
OWL-ViT	24.3	350
Grounding DINO	4.5	2357
YOLO-World	84.6	715

Takeaway

Accuracy and throughput do not rise together. Pick the model by deployment budget.

RefCOCO Grounding Results



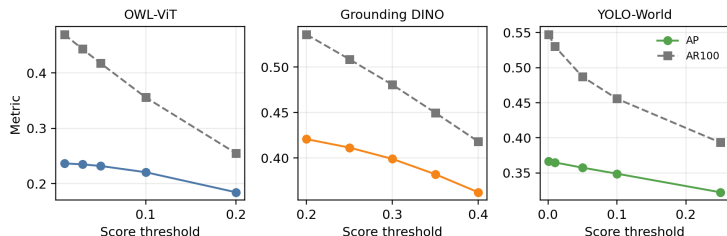
Why this is stricter

RefCOCO evaluates the exact referred instance, not just the category name. The final run uses every expression, so it is much harder than a 100-row diagnostic.

Numbers

- Grounding DINO: 51.1% / 45.6% / 0.514
- OWL-ViT: 42.5% / 34.2% / 0.423
- YOLO-World: 41.4% / 36.2% / 0.422

Threshold Sensitivity and NMS



Main observation

Thresholds are model-specific.
Cleaner visualizations usually mean lower recall, not better AP.

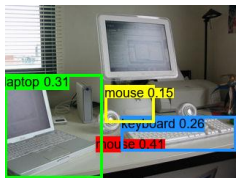
AP drop

OWL-ViT	0.237 → 0.184
Grounding DINO	0.421 → 0.362
YOLO-World	0.366 → 0.322

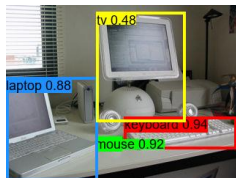
NMS diagnostic

OWL-ViT 100-image subset: AP 0.336
→ 0.323; AR100 0.520 → 0.475.

Qualitative Examples and Failure Modes



OWL-ViT workspace



Grounding DINO workspace



OWL-ViT traffic



Grounding DINO traffic

Two comparisons

- Workspace: both models detect the main objects.
- Traffic: small or occluded objects are harder.
- Grounding DINO is usually tighter.

Common errors

- duplicate boxes
- nearby-object confusion
- spatial relation failures

Limitations and Takeaways

Limitations

- No fine-tuning or learned calibration.
- COCO and RefCOCO are still curated benchmarks, not open-world data.
- Runtime depends on hardware, precision, and input resolution.

Future work

- LVIS or ODinW for longer-tail categories
- Flickr30K Entities for caption-style grounding
- Prompt paraphrase study for robustness

Final takeaways

- Grounding DINO: best accuracy on both COCO and RefCOCO.
- YOLO-World: best speed / efficiency trade-off.
- OWL-ViT: simplest baseline and low-memory reference.
- The project is fully reproducible and based on full-split results.

One-sentence defense

We did not just run a demo. We built a unified, reproducible benchmark for open-vocabulary detection and grounding.

Thank you

Questions are welcome.

Backup: Exact Full-Split Numbers

COCO val2017

Model	AP	AP50	AP75	AR100	FPS
OWL-ViT	0.237	0.385	0.249	0.469	24.3
Grounding DINO	0.421	0.557	0.457	0.536	4.5
YOLO-World	0.366	0.510	0.398	0.547	84.6

RefCOCO val

Model	Acc@0.5	Acc@0.75	Mean IoU	FPS
OWL-ViT	0.425	0.342	0.423	29.9
Grounding DINO	0.511	0.456	0.514	3.1
YOLO-World	0.414	0.362	0.422	115.1

Diagnostic NMS check

OWL-ViT 100-image subset: AP 0.336 $\rightarrow$ 0.323
and AR100 0.520 $\rightarrow$ 0.475 after class-wise NMS.

Backup: Reproducibility Checklist

- Server rerun script: `bash scripts/run_full_suite_4gpu_server.sh`
- Shared configuration block: `COCO_MAX_IMAGES=0, REFCOCO_MAX_ROWS=0, REFCOCO_EXPRESSION_MODE=all`
- RefCOCO manifest fallback: local Hugging Face parquet snapshot under `data/refcoco/hf_snapshot`
- Final artifact directories: `outputs/server_2026_06/` and `logs/server_2026_06/`
- Paper format: IEEE conference style, fully compiled locally before submission